

Hospital Department Count Using Hadoop MapReduce

1. Aim

To implement and demonstrate a Hadoop MapReduce program for counting the number of patients in each hospital department using a hospital patient dataset stored in HDFS (Hadoop Distributed File System).

2. Problem Statement

A hospital maintains patient information such as Patient ID, Name, Age, Department, Diagnosis, and Admission Days in a CSV file. The objective is to:

- Store the hospital dataset in HDFS.
- Process the dataset using Hadoop MapReduce.
- Count the number of patients in each department.
- Display the department-wise patient count.
- Perform additional analysis such as finding maximum, minimum, total, and filtered department counts.

3. What is Hadoop?

Hadoop is an open-source framework used for distributed storage and processing of large datasets. Instead of storing and processing all data on a single system, Hadoop distributes the data and processing across multiple machines. It is mainly used for:

- Storing large amounts of data
- Processing large datasets
- Distributed computing
- Parallel processing
- Handling data efficiently

In this practical, Hadoop is used to process the hospital patient dataset.

4. What is HDFS?

HDFS stands for Hadoop Distributed File System. It is the storage component of Hadoop. In this practical, the input file is stored at:

```
/hospital/input/hospital_data.csv
```

The output is stored at:

```
/hospital/output/
```

Main HDFS components

- **NameNode** - Manages HDFS metadata and keeps information about files and blocks.
- **DataNode** - Stores the actual data.
- **Secondary NameNode** - Helps maintain checkpoints of NameNode information.

5. What is YARN?

YARN stands for Yet Another Resource Negotiator. It is responsible for resource management and job scheduling in Hadoop.

Main YARN components

- **ResourceManager** - Manages resources of the Hadoop cluster and coordinates applications/jobs.
- **NodeManager** - Manages resources and tasks on an individual node.

In this practical, YARN was started using:

```
start-yarn.sh
```

6. What is MapReduce?

MapReduce is a Hadoop programming model used to process data in parallel. It has the following stages:

```
Input -> MAP -> SHUFFLE -> REDUCE -> Output
```

Map: The Mapper reads each patient record and extracts the department. For example, the record

```
P001,Ravi,45,Cardiology,Heart_Disease,5
```

produces:

```
Cardiology    1
```

Shuffle: Hadoop groups records having the same department, e.g. Cardiology -> [1,1,1,1].

Reduce: The Reducer adds the values, e.g. Cardiology -> 4. The final output contains the count for every department.

7. Components Used

Component	Purpose
Hadoop	Distributed data processing framework
HDFS	Stores input and output files
NameNode	Manages HDFS metadata
DataNode	Stores actual data
SecondaryNameNode	Maintains checkpoints
YARN	Resource management
ResourceManager	Manages cluster resources
NodeManager	Manages node resources
MapReduce	Processes the hospital data
Mapper	Generates department-count pairs
Reducer	Calculates department totals

8. Technology Stack

Technology	Use
Ubuntu/Linux	Operating environment
Apache Hadoop	Big-data framework
HDFS	Distributed file storage
YARN	Resource management
MapReduce	Data processing

Technology	Use
Java	Hadoop MapReduce application
Hadoop JAR	Executes the MapReduce application
Terminal	Execute Hadoop commands
AWK	Filter and analyze data
grep	Search records
sort	Sort results
head	Display beginning records
tail	Skip/display records

The MapReduce JAR used is `hospital-mapreduce.jar` and the main class is `HospitalDepartmentCount`.

9. Software Version

The exact Hadoop and Java versions are not visible in the screenshots, so they are not stated here. Obtain them using:

```
hadoop version
java -version

OS      : Ubuntu Linux
Hadoop  : [version obtained from hadoop version]
Java    : [version obtained from java -version]
```

10. Installation and Configuration

The Hadoop environment was configured on Ubuntu, including YARN and MapReduce-related settings. The file modified was:

```
$HADOOP_HOME/etc/hadoop/yarn-site.xml
```

The property `yarn.nodemanager.delete.debug-delay-sec` was set to **600**. YARN was then restarted using:

```
stop-yarn.sh && sleep 3 && start-yarn.sh
```

This stops YARN, waits 3 seconds, and starts YARN again. The services were checked using `jps`. The expected running processes were:

```
NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps
```

This confirms that both HDFS and YARN were running.

11. Starting Hadoop Services

Start HDFS

```
start-dfs.sh
```

Starts the HDFS services: NameNode, DataNode, SecondaryNameNode.

Start YARN

```
start-yarn.sh
```

Starts ResourceManager and NodeManager.

Check running services

```
jps
```

Verifies that the Hadoop Java processes are running.

12. Input File

The input file `hospital_data.csv` was stored in HDFS at `/hospital/input/hospital_data.csv`. Its columns are: Patient_ID, Name, Age, Department, Diagnosis, Admission_Days. Example records:

```
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
P004,Meena,28,Pediatrics,Infection,3
```

The dataset contains 20 patient records plus one header, giving 21 input records. The MapReduce counters confirm: Map input records = 21 and Map output records = 20.

13. Command to Display Input File

```
hdfs dfs -cat /hospital/input/hospital_data.csv
```

Displays the contents of the hospital CSV file stored in HDFS.

14. Command to Display First 3 Patients

```
hdfs dfs -cat /hospital/input/hospital_data.csv | tail -n +2 | head -3
```

`tail -n +2` skips the header; `head -3` displays the first three patient records. Output:

```
P001,Ravi,45,Cardiology,Heart_Disease,5
P002,Anu,32,General_Medicine,Fever,2
P003,Kiran,60,Orthopedics,Fracture,7
```

15. Searching for a Patient

```
hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $1=="P015"'
```

Output:

```
P015,Anil,63,Orthopedics,Fracture,9
```

Here `$1` = Patient_ID and `$4` = Department.

16. Display Patient ID and Department

```
hdfs dfs -cat /hospital/input/hospital_data.csv | \
  awk -F',' ' $1=="P015"{print "Patient ID:",$1,"Department:",$4}'
```

Output:

```
Patient ID: P015 Department: Orthopedics
```

This demonstrates field extraction using AWK.

17. Find Cardiology Patients

```
hdfs dfs -cat /hospital/input/hospital_data.csv | awk -F',' ' $4=="Cardiology" | head -3
```

Output:

```
P001,Ravi,45,Cardiology,Heart_Disease,5
```

P007,Suresh,67,Cardiology,Hypertension,6
P013,Vijay,42,Cardiology,Heart_Disease,6

Here \$4 represents the Department field.

18. Executing MapReduce

```
hadoop jar hospital-mapreduce.jar HospitalDepartmentCount \  
/hospital/input/hospital_data.csv /hospital/output
```

- `hadoop jar` - runs a Hadoop Java application.
- `hospital-mapreduce.jar` - the MapReduce application.
- `HospitalDepartmentCount` - the main class.
- `/hospital/input/hospital_data.csv` - the input file.
- `/hospital/output` - the output directory.

19. MapReduce Process

```
hospital_data.csv -> Mapper -> (Department, 1)  
-> Shuffle (group departments) -> Reducer -> Department count
```

For example, four pairs of Cardiology -> 1 are reduced to Cardiology -> 4.

20. MapReduce Execution Result

The MapReduce job completed successfully, showing *map 100% reduce 100%*. The important counters were:

Counter	Value
Map input records	21
Map output records	20
Reduce input records	20
Reduce output records	6

21. Output

The output was stored in `/hospital/output/part-r-00000`.

```
hdfs dfs -cat /hospital/output/part-r-00000
```

```
Cardiology      4  
Dermatology    3  
General_Medicine 4  
Neurology      3  
Orthopedics    3  
Pediatrics     3
```

22. Result Analysis

Maximum count

```
hdfs dfs -cat /hospital/output/part-r-00000 | sort -k2,2nr | head -1
```

Result: Cardiology 4. However, General_Medicine also has 4, so the maximum is shared by two departments.

Minimum count

The minimum count is 3. Departments with 3 patients: Dermatology, Neurology, Orthopedics, Pediatrics.

Total patients

```
hdfs dfs -cat /hospital/output/part-r-000000 | awk '{sum += $2} END {print sum}'
```

Result: **20**

Departments with more than 3 patients

```
Cardiology      4
General_Medicine 4
```

23. Final Result Table

Department	Number of Patients
Cardiology	4
General_Medicine	4
Dermatology	3
Neurology	3
Orthopedics	3
Pediatrics	3
Total	20

24. Outcome

The Hadoop MapReduce program was successfully implemented and executed. The experiment demonstrated:

1. Starting HDFS and YARN services.
2. Verifying Hadoop services using jps.
3. Storing and reading the hospital dataset from HDFS.
4. Processing the dataset using MapReduce.
5. Performing Map, Shuffle and Reduce operations.
6. Calculating department-wise patient counts.
7. Using Linux commands such as awk, grep, sort, head, and tail for data analysis.
8. Successfully generating the final department-wise patient count.

The job completed successfully with 20 patient records processed and 6 department-wise results generated.

25. Conclusion

The experiment successfully demonstrated the use of Hadoop HDFS, YARN and MapReduce for processing hospital data. HDFS was used for storing the input dataset, YARN managed the Hadoop resources, and MapReduce processed the patient records to calculate department-wise counts. The final result correctly identified the number of patients in each hospital department, with a total of 20 patients.